1)- while using ceil and floor functions always use 'double'

Ex. ceil((double)k/(double)i)
floor((double)p/(double j))

2)- To find the maximum and minimum element within an vector use *max_element(arr.begin(), arr.end()).
*min_element(arr.begin(), arr.end()).

3)- To declare a vector of size 'n' and whose all elements are initialized to zero, the syntax is-

vector<int>nums(n,0);

4) while using comparators:-

```
bool comp(int x, int y) {
    return x>y;
}
```

(Now we will use it to sort the array in discanding order).

i) Syntax for the plain array:- (a[] = {1,3,5,9,8})

```
int n= sizeof(a)/sizeof(a[0])
sort(a, a+n, comp)
```

ii) Syntax for vector:- (vector<int>nums)

```
sort(nums.begin(), nums.end(), comp);
```

→ to place the median element at its correct position-

n is the length of vector

```
nth_element(v.begin(), v.begin+n/2, v.end())
```

Note: i) To take a defaut value to all indices of an array use-

```
int arr[n]={-1};
```

ii) But, In vector use vector<int>v(n,-1);

i) To delete an element at index i in vector use.

`v.erase(v.begin()+i)`

2) To replace a character in a string with another character –

replace(str.begin(), str.end(), oldchar, new_char).

3) In string to find the length use .str.length().

4) To insert the characters of set in vector use –

vector<int>(st.begin(), st.end())

st → name of set.

5).* i) (char)3 means the character whose ASCII value is 3.

ii) (char)('0'+3) gives '3'; because '0' has ASCII value code 48, & 48+3 = 51.

(means on adding a character with int the resultant is ASCII (int form) value of the +int)

ii) On adding a character to a int the resultant is character by default.

On adding two characters we get a character.

iii) Ascii value of 'a' is 97 & 'z' is 122, and 'A' is 65 and 'Z' is 90.

iv) To make a string of x (no. of '4') → '4' → string((size_t)x, '4')

Note. To transform a number n to m will will require the same number of moves, it will not depend on the way of transformation.

a) → To print a space before a number/string —
cout << setw(5) << 4; (will print -4)
↳ leaving 5 space

Cout << setw(2) << setfill('0') << hours;
↳ it will add '0' before hours if the length of 'hours' is less than 2. i.e if hours = 2, it will print 02, but for 10, it will print 10.

Q → To write the 'number' till n decimal places -
cout << fixed << setprecision(n) << number;

Note - For Sets -

i) In sets to find the last element —
auto it = st.rbegin();
last element = *it;

ii) Suppose set contains {1, 2, 3, 4, 5}, we want to find the before and after element of 4, then use it —

auto it = st.find(4)
auto before = prev(it)
→ int before_element = *before.

auto after = next(it);
int after_element = *after.

→ auto it = st.insert(x).first (st)

↳ It will insert 'x' into set and will store the iterator to 'x' in 'it'.

→ In multiset (mt) to delete an element say 'a', use the following syntax -

`mt.erase(mt.find(a))` → will delete the first occurance of a.

`mt.erase(a)` → will delete the all occurances of a.

Concept of Sliding Window:-

1) There are 4-patterns -

1) Constant Window -

Ex. We have an $= [1, -2, 0, 1, 2, 3, 4]$ (l at 1, r at 1)
and k=4, we have to find the maximum sum picking k-consecutive integers.
To solve this we take two pointer l & r and start moving them.

2) longest Subarray/Substring -

Ex. longest subarray with sum ≤ k.

Ex. a[] = [2, 5, 1, 7, 10], k=14. (l at 2, r at 5)

```
while(r<n){
        sum = sum+arr[r]
        while(sum>k){
            sum = sum-arr[l]
            l++;
        }
        if(sum≤k){
            maxlen = max(maxlen, r-l+1);
            r=r+1;
        }
}
```

T.C = O(2N)

we can make it O(N) as follows - Just use 'if' instead of inner while loop. (means we will shrink only when sum>k, just once, and will not go below maxlen, which we obtained before).

3)- No. of Subarrays with some Condition -

↳ These are solved using the pattern 2 discussed above.

→ suppose we want to find the number of subarrays with sum = k.
then find no. of subarrays with sum ≤ k = x
" " " " " " ≤ k-1 = y
then no. of subarrays with sum = k ⇒ x-y.

4) Finding the shortest window/length with the given condition.

Ques-

1423. Maximum Points You Can Obtain from Cards

Solved

Medium | Topics | Companies | Hint

There are several cards **arranged in a row**, and each card has an associated number of points. The points are given in the integer array `cardPoints`.

In one step, you can take one card from the beginning or from the end of the row. You have to take exactly `k` cards.

Your score is the sum of the points of the cards you have taken.

Given the integer array `cardPoints` and the integer `k`, return the *maximum score* you can obtain.

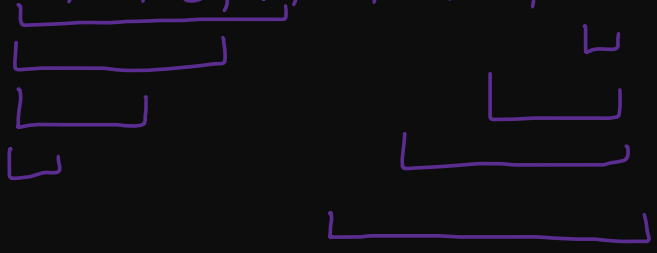
Example 1:

C++ | Auto

```cpp
class Solution {
public:
    int maxScore(vector<int>& cardPoints, int k) {
        int l = k-1;
        int n = cardPoints.size();
        int r = n-1;
        int lsum = 0;
        int rsum = 0;
        for(int i = 0; i<k; i++){
            lsum = lsum + cardPoints[i];
        }
        int maxsum = lsum;
        while(lsum>0){
            lsum -= cardPoints[l];
            rsum += cardPoints[r];
            l--;
            r--;
            maxsum =  max(maxsum,lsum+rsum);
        }
        return maxsum;
```

Saved Ln 20, Col 23

Solution- 6, 2, 3, 4, 7, 1, 2, 1, k = 4



first taken all the the elements from left then shrink that window and taken 1, element from right, so, on.....

Ques

3. Longest Substring Without Repeating Characters

Solved

Medium | Topics | Companies | Hint

Given a string `s`, find the length of the **longest** **substring** without duplicate characters.

```cpp
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        int n = s.length();
        if(n==0)return 0;
        int l = 0;
        int r = 0;
        map<char,int>vis;
        int length = 0;
        int maxlength = 0;
        while(r<=n-1){
            vis[s[r]]++;
            if(vis[s[r]]==1){
                length += 1;
                maxlength = max(maxlength,length);
                r++;
            }
            else{
                for(int i = l; i<n; i++){
                    vis[s[i]]--;
                    if(vis[s[r]]<=1){
                        l = i+1;
                        length = r-l+1;
                        break;
                    }
                }
                r++;
            }
        }
        return maxlength;
```

Ques-

1004. Max Consecutive Ones III

Solved

Medium | Topics | Companies | Hint

Given a binary array `nums` and an integer `k`, return *the maximum number of consecutive* `1`*'s in the array if you can flip at most* `k` `0`'s.

```cpp
    int longestOnes(vector<int>& nums, int k) {
        int l = 0;
        int r = 0;
        int n = nums.size();
        if(n==1 && k==0 && nums[n-1]==0)return 0;
        if(n==1 && k==0 && nums[n-1]==1 || n==1 && k==1 && nums[n-1]==0)return 1;
        int length = 0;
        int maxlength = 0;
        int k1 = k;
        while(r<=n-1){
            if(nums[r]==1){
                r++;
                length += 1;
                maxlength = max(length,maxlength);
            }
            else if(nums[r]==0 && k>0){
                k--;
                r++;
                length += 1;
                maxlength = max(length, maxlength);
            }
            else if(nums[r]==0 && k<=0){
                length = 0;
                k = k1;
                l++;
                r = l;
            }
        }
        return maxlength;
    }
```

T.C = O(N).

Ques-

904. Fruit Into Baskets

Attempted

Medium | Topics | Companies

You are visiting a farm that has a single row of fruit trees arranged from left to right. The trees are represented by an integer array `fruits` where `fruits[i]` is the **type** of fruit the `i`th tree produces.

You want to collect as much fruit as possible. However, the owner has some strict rules that you must follow:

- You only have **two** baskets, and each basket can only hold a **single type** of fruit. There is no limit on the amount of fruit each basket can hold.
- Starting from any tree of your choice, you must pick **exactly one fruit** from **every** tree (including the start tree) while moving to the right. The picked fruits must fit in one of your baskets.
- Once you reach a tree with fruit that cannot fit in your baskets, you must stop.

Given the integer array `fruits`, return *the **maximum** number of fruits you can pick.*

Example 1:

```
Input: fruits = [1,2,1]
Output: 3
Explanation: We can pick from all 3 trees.
```

Example 2:

```
Input: fruits = [0,1,2,2]
Output: 3
Explanation: We can pick from trees [1,2,2].
If we had started at the first tree, we would only pick from
trees [0,1].
```

```cpp
class Solution {
public:
    int totalFruit(vector<int>& fruits) {
        int n = fruits.size();
        int r = 0;
        int l = 0;
        unordered_map<int,int>mpp;
        long long length = 0;
        long long maxlength = 0;
        while(r<n){
            mpp[fruits[r]]++;
            if(mpp.size()<=2){
                length++;
                maxlength = max(length,maxlength);
                r++;
            }
            else{
                mpp.clear();
                l++;
                r = l;
                length = 0;
            }
        }
        return maxlength;
    }
};
```

This will give TLE due to 'mpp.clear' as it takes $O(n^2)$ instead just erase the l^{th} fruit from map. so use mpp.erase(fruit[l])

Second approach -

```cpp
class Solution {
public:
    int totalFruit(vector<int>& fruits) {
        int n = fruits.size();
        int r = 0;
        int l = 0;
        unordered_map<int,int>mpp;
        long long length = 0;
        long long maxlength = 0;
        while(r<n){
            mpp[fruits[r]]++;
            if(mpp.size()<=2){
                length++;
                maxlength = max(length,maxlength);
                r++;
            }
            else{
                mpp.erase(fruits[l]);
                l++;
                r = l;
                length = 0;
            }
        }
        return maxlength;
    }
};
```

→ not neussary.

optimization- Don't move r to l, instead check if mpp.size >2 if yes then move l, decrease the length and if mpp[fruits[l]] = 0, then erase it.

Ques.

1358. Number of Substrings Containing All Three Characters

Solved

Medium | Topics | Companies | Hint

Given a string `s` consisting only of characters *a*, *b* and *c*.

Return the number of substrings containing **at least** one occurrence of all these characters *a*, *b* and *c*.

Example 1:

> **Input:** s = "abcabc"
> **Output:** 10
> **Explanation:** The substrings containing at least one occurrence of the characters *a*, *b* and *c* are "abc", "abca", "abcab", "abcabc", "bca", "bcab", "bcabc", "cab", "cabc" and "abc" (**again**).

Example 2:

C++ | Auto

```cpp
class Solution {
public:
    int numberOfSubstrings(string s) {
        int n = s.size();
        int l = 0;
        int r = 0;
        int count = 0;
        vector<int>vis(3,-1);
        for(int i = 0; i<n; i++){
            vis[s[i]-'a'] =  i;
            if(vis[0]!=-1 && vis[1]!=-1 && vis[2]!=-1){
                int m1 = min(vis[0], vis[1]);
                int m = min(vis[2], m1);
                l = m;
                count += l+1;
            }
        }
        return count;
    }
};
```

Saved | Ln 15, Col 30

Testcase | Test Result

Intution- S = bbacba first, we will find a minimum window which contains all the three characters

Ex. b b a c b a , now no. of substring

0 1 2 3 4 5

using this window is, — just expand the window till left — so, total no. of substring = lowest index of window + 1.

Ques-

Problem List | Run | Submit

Description | Editorial | Solutions | Submissions

424. Longest Repeating Character Replacement

Medium | Topics | Companies

You are given a string `s` and an integer `k`. You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most `k` times.

Return *the length of the longest substring containing the same letter you can get after performing the above operations.*

Example 1:

```
Input: s = "ABAB", k = 2
Output: 4
Explanation: Replace the two 'A's with two 'B's or vice versa.
```

Example 2:

```
Input: s = "AABABBA", k = 1
Output: 4
Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".
The substring "BBBB" has the longest repeating
```

Code

C++ | Auto

```cpp
class Solution {
public:
    int characterReplacement(string s, int k) {
        int n = s.length();
        int l = 0;
        int r = 0 ;
        int count = 0;
        vector<int>v(26,0);
        int maxcount = 0;
        int maxfreq = 0;
        while(r<n){
            v[s[r]-'A']++;
            maxfreq = max(maxfreq, v[s[r]-'A']);
            if(r-l+1-maxfreq<=k){
                maxcount = max(maxcount, r-l+1);
                r++;
            }
            else{
                v[s[l]-'A']--;
                l++;
                r++;
            }
        }
        return maxcount;
    }
};
```

Saved

* ⇒ as there is no point in finding in the reduced window length.

Ques.

930. Binary Subarrays With Sum

Medium | Topics | Companies

Given a binary array `nums` and an integer `goal`, return *the number of non-empty* ***subarrays*** *with a sum* `goal`.

A **subarray** is a contiguous part of the array.

Here we have zero, so we will have dilema whether to move l or r. at some point of time.

Ans. No. of subarrays with sum $\leq k \longrightarrow x$.

No of subarrays " " $\leq k-1 \longrightarrow y$.

No. of subarrays with sum $= k \longrightarrow x-y$.

Eg. 1 0 1 1 0 0, goal = 2. So, subarrays with sum $\leq$ goal can be. [ ] } → all these can be answer So, r-l+1 has been summed up.

```cpp
public:
    int numSubarraysWithSum(vector<int>& nums, int goal) {
        int n = nums.size();
        int l = 0;
        int r = 0;
        long long sum = 0;
        long long sum2 = 0;
        int count1 = 0;
        int count2 = 0;
        while(r<n){
            sum = sum + nums[r];
            if(sum<=goal){
                count1 += r-l+1;
                r++;
            }
            else if(sum>goal){
                while(sum>goal && l<=r){  // ✅ l<=r to avoid over-subtracting
                    sum = sum - nums[l];
                    l++;
                }
                if(sum<=goal){
                    count1 += r-l+1;
                }
                r++;
            }
        }
        l=0;
        r=0;
        if(goal>0){
            while(r<n){
                sum2 = sum2 + nums[r];
                if(sum2<=goal-1){
                    count2 += r-l+1;
                    r++;
                }
                else if(sum2>goal-1){
                    while(sum2>goal-1 && l<=r){  // ✅ l<=r to stay safe
                        sum2 = sum2 - nums[l];
                        l++;
                    }
                    if(sum2<=goal-1){
                        count2 += r-l+1;
                    }
                    r++;
                }
            }
        }
        return count1-count2;
    }
};
```

Ques-

1248. Count Number of Nice Subarrays

Solved

Medium | Topics | Companies | Hint

Given an array of integers `nums` and an integer `k`. A continuous subarray is called **nice** if there are `k` odd numbers on it.

Return *the number of **nice** sub-arrays.*

→ This question is just like just above question, we will transform the arrays with odd numbers as 1 & even num as '0' & will solve like above.

Example 1:

> **Input:** nums = [1,1,2,1,1], k = 3
> **Output:** 2
> **Explanation:** The only sub-arrays with 3 odd numbers are [1,1,2,1] and [1,2,1,1].

```
2  private:
3      int lessthan_k(vector<int>&nums, int k){
4          if(k<0)return 0;
5          int count = 0;
6          int sum = 0;
7          int l = 0;
8          int r = 0;
9          int n = nums.size();
10         while(r<n){
11             sum = sum + nums[r];
12             if(sum<=k){
13                 count += r-l+1;
14                 r++;
15             }
16             if(sum>k){
17                 while(sum>k && l<=r){
18                     sum = sum - nums[l];
19                     l++;
20                 }
21                 if(sum<=k){
22                     count += r-l+1;
23                 }
24                 r++;
25             }
26
27         }
28         return count;
29     }
```

```cpp
public:
    int numberOfSubarrays(vector<int>& nums, int k) {
        int n = nums.size();
        for(int i = 0; i<n; i++){
            if(nums[i]%2==0){
                nums[i]=0;
            }
            else{
                nums[i] = 1;
            }
        }
        return funct(nums,k)-funct(nums,k-1);
    }
};
```

could have done without this also in above code just write sum=sum+num[r]%2.

Ques -

992. Subarrays with K Different Integers

Solved

Hard | Topics | Companies | Hint

Given an integer array `nums` and an integer `k`, return *the number of **good subarrays** of* `nums`.

A **good array** is an array where the number of different integers in that array is exactly `k`.

- For example, `[1,2,3,1,2]` has `3` different integers: `1`, `2`, and `3`.

A **subarray** is a **contiguous** part of an array.

Here also we will face the dilema at a point that whether to move l or l, so, will solve using above methods.

Ans :-

```cpp
class Solution {
private:
    int less_than_k_distinct(vector<int>&nums,int k){
        int n = nums.size();
        int l = 0;
        int r = 0;
        map<int,int>mpp;
        int count = 0;
        while(r<n){
            mpp[nums[r]]++;
            if(mpp.size()<=k){
                count += r-l+1;
                r++;
            }
            else if(mpp.size()>k){
                while(mpp.size()>k && l<=r){
                    mpp[nums[l]]--;
                    if(mpp[nums[l]]==0){
                        mpp.erase(nums[l]);
                    }
                    l++;
                    if(mpp.size()<=k){
                        count += r-l+1;
                    }
                }
                r++;
            }
        }
        return count;
    }
public:
    int subarraysWithKDistinct(vector<int>& nums, int k) {
        return less_than_k_distinct(nums,k)-less_than_k_distinct(nums,k-1);
    }
};
```

76. Minimum Window Substring

Hard | Topics | Companies | Hint

Given two strings `s` and `t` of lengths `m` and `n` respectively, return *the* ***minimum window substring*** *of* `s` *such that every character in* `t` *(**including duplicates**) is included in the window*. If there is no such substring, return *the empty string* `""`.

The testcases will be generated such that the answer is **unique**.

Example 1:

> **Input:** s = "ADOBECODEBANC", t = "ABC"
> **Output:** "BANC"
> **Explanation:** The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.

We have taken a count variable to see whether all the characters of 't' are covered. First we take a hashmap and put all the characters of string t with their freq.. Now, as 'r' move through the string S we decrease the frequency of that character untill all the characters in 't' are covered, (i.e count = t.size()). Then we move 'l' and increase the freq. of character, and if hash[S[l]] > 0 then we stop there, as we will reach the min boundary of that window where all the characters of t will be present.

```
public:
    string minWindow(string s, string t) {
        int minLen = INT_MAX;
        int sIndex = -1;
        int hash[256]={0};
        for(char c : t){
           hash[c]++;
        }
        int count =0;
        int l=0,r=0;
        while(r<s.size()){
           if(hash[s[r]]>0){
               count++;
           }
           hash[s[r]]--;
           while(count == t.size()){
               if(r-l+1<minLen){
                   minLen = r-l+1;
                   sIndex = l;
               }
               hash[s[l]]++;
               if (hash[s[l]] > 0) {
                   count--;
               }
               l++;
           }
           r++;
        }
        return (sIndex == -1) ? "" : s.substr(sIndex, minLen);
    }
};
```

Ques 1 - (Brute)

1352. Product of the Last K Numbers

Solved

Medium | Topics | Companies | Hint

Design an algorithm that accepts a stream of integers and retrieves the product of the last `k` integers of the stream.

Implement the `ProductOfNumbers` class:

- `ProductOfNumbers()` Initializes the object with an empty stream.
- `void add(int num)` Appends the integer `num` to the stream.
- `int getProduct(int k)` Returns the product of the last `k` numbers in the current list. You can assume that always the current list has at least `k` numbers.

The test cases are generated so that, at any time, the product of any contiguous sequence of numbers will fit into a single 32-bit integer without overflowing.

C++ Auto

```cpp
class ProductOfNumbers {
public:
    ProductOfNumbers() {

    }
    vector<int>v;
    int n;
    void add(int num) {
        v.push_back(num);
        n = v.size();
    }

    int getProduct(int k) {
        int count = 0;
        int product = 1;
        for(int i = n-1; i>=n-k; i--){
            product = product*v[i];
        }
        return product;
    }
};

/**
```

optimal

C++ Auto

```cpp
class ProductOfNumbers {
public:
    ProductOfNumbers() {

    }
    vector<int>v;
    int product = 1;
    void add(int num) {
       if(num!=0){
        product = product*num;
        v.push_back(product);
       }
       else{
        v.clear();
        product = 1;
       }
    }

    int getProduct(int k) {
       if(k>v.size()){
        return 0;
       }
       else if(k==v.size()){
        return v[v.size()-1];
       }
       else{
        return v[v.size()-1]/(v[v.size()-k-1]);
       }
    }
};
```

1358. Number of Substrings Containing All Three Characters

Solved

Medium | Topics | Companies | Hint

Given a string `s` consisting only of characters *a*, *b* and *c*.

Return the number of substrings containing **at least** one occurrence of all these characters *a*, *b* and *c*.

Example 1:

> **Input:** s = "abcabc"
> **Output:** 10
> **Explanation:** The substrings containing at least one occurrence of the characters *a*, *b* and *c* are "abc", "abca", "abcab", "abcabc", "bca", "bcab", "bcabc", "cab", "cabc" and "abc" (**again**).

C++ Auto

```cpp
class Solution {
public:
    int numberOfSubstrings(string s) {
        int left = 0;
        int count = 0;
        int n = s.length();
        int a = 0, b = 0, c = 0;

        for (int right = 0; right < n; ++right) {
            if (s[right] == 'a') a++;
            if (s[right] == 'b') b++;
            if (s[right] == 'c') c++;

            while (a > 0 && b > 0 && c > 0) {
                count += n - right; // Count the substrings
                if (s[left] == 'a') a--;
                if (s[left] == 'b') b--;
                if (s[left] == 'c') c--;
                left++;
            }
        }

        return count;
```

Sliding window Concept -

3306. Count of Substrings Containing Every Vowel and K Consonants II

Solved

Medium | Topics | Companies | Hint

You are given a string `word` and a **non-negative** integer `k`.

Return the total number of substrings of `word` that contain every vowel (`'a'`, `'e'`, `'i'`, `'o'`, and `'u'`) **at least** once and **exactly** `k` consonants.

Example 1:

> **Input:** `word = "aeioqq", k = 1`
>
> **Output:** `0`
>
> **Explanation:**

```
class Solution {
public:
    bool isvowel(char c ){
        return c =='a' || c == 'e' || c ==  'i' || c == 'o' ||  c == 'u';
    }
    long long atleastk(string &word, int k ){
        int n = word.size();
        map<char,int>vowelmap;
        int left = 0;
        int consonent = 0;
        long long ans = 0;
        for(int right = 0; right < n; right++){
```

```
            if(isvowel(word[right])){
                vowelmap[word[right]]++;
            }
            else{
                consonent++;
            }
            while(vowelmap.size()==5 && consonent >=k){
                ans += n-right;
                if(isvowel(word[left])){
                    vowelmap[word[left]]--;
                    if(vowelmap[word[left]]==0){
                        vowelmap.erase(word[left]);
                    }

                }
                else{
                    consonent--;
                }
                left++;
            }
        }
        return ans;
    }
    long long countOfSubstrings(string word, int k) {
        return atleastk(word,k)-atleastk(word,k+1);
    }
};
```

3191. Minimum Operations to Make Binary Array Elements Equal to One I

Solved

Medium | Topics | Companies | Hint

You are given a binary array `nums`.

You can do the following operation on the array **any** number of times (possibly zero):

- Choose **any** 3 **consecutive** elements from the array and **flip all** of them.

Flipping an element means changing its value from 0 to 1, and from 1 to 0.

Return the **minimum** number of operations required to make all elements in `nums` equal to 1. If it is impossible, return -1.

Brute –

```cpp
class Solution {
public:
    int minOperations(vector<int>& nums) {
        int n = nums.size();
        int low = 0;
        int high = 2;
        int count = 0;
        while(low<n && high < n){
            if(nums[low]==0){
                for(int i = low; i<=high; i++){
                    nums[i] = nums[i]^1;
                }
                low++;
                high++;
                count++;
            }
            else{
                low++;
                high++;
            }
        }
        for(int j = 0; j<n; j++){
            if(nums[j]==0){
                return -1;
            }
        }
        return count;
    }
```

Sliding Window approach —

```cpp
class Solution {
public:
    int minOperations(vector<int>& nums) {
        int count = 0;
        for (int i = 2; i < nums.size(); i++) {
            // only looking forward to the last element
            if (nums[i - 2] == 0) {
                count++;
                // flip i-2 and i-1 and i
                nums[i - 2] ^= 1;
                nums[i - 1] ^= 1;
                nums[i] ^= 1;
            }
        }
        int sum = accumulate(nums.begin(), nums.end(), 0);
        if (sum == nums.size()) return count;
        return -1;
    }
};
```

2529. Maximum Count of Positive Integer and Negative Integer

Solved

Easy | Topics | Companies | Hint

Given an array `nums` sorted in **non-decreasing** order, return *the maximum between the number of positive integers and the number of negative integers.*

- In other words, if the number of positive integers in `nums` is `pos` and the number of negative integers is `neg`, then return the maximum of `pos` and `neg`.

Note that `0` is neither positive nor negative.

Example 1:

Input: nums = [-2,-1,-1,1,2,3]
Output: 3

C++ | Auto

```cpp
class Solution {
public:
    int maximumCount(vector<int>& nums) {
        int neg_count = binarySearch(nums, 0);
        int pos_count = nums.size() - binarySearch(nums, 1);
        return max(neg_count, pos_count);
    }

private:
    int binarySearch(vector<int>& nums, int target) {
        int left = 0, right = nums.size() - 1, result = nums.size();

        while (left <= right) {
            int mid = (left + right) / 2;
            if (nums[mid] < target) {
                left = mid + 1;
            } else {
                result = mid;
                right = mid - 1;
            }
        }

        return result;
```

3356. Zero Array Transformation II

Solved

Medium | Topics | Companies | Hint

You are given an integer array `nums` of length `n` and a 2D array `queries` where `queries[i] = [l_i, r_i, val_i]`.

Each `queries[i]` represents the following action on `nums`:

- Decrement the value at each index in the range `[l_i, r_i]` in `nums` by **at most** `val_i`.
- The amount by which each value is decremented can be chosen **independently** for each index.

A **Zero Array** is an array with all its elements equal to 0.

Return the **minimum** possible **non-negative** value of `k`, such that after processing the first `k` queries in **sequence**, `nums` becomes a **Zero Array**. If no such `k` exists, return -1.

A simple approach would be to iterate through each query, applying the updates directly to nums and checking whether all elements have become zero. However, given the constraints where both nums and queries can be as large as 105, this approach is too slow. Each query might require traversing the entire array, leading to an impractical time complexity.

To optimize this, we need a more efficient way to apply queries to nums. Instead of modifying each element individually, we can take advantage of a **difference array**. This technique allows us to apply a range update in constant time. The key idea is to store the changes at the boundaries of the range rather than updating every element inside it. For a query [left,right,val], we add val at index left, and subtract val at index right + 1. When we later compute the prefix sum of this difference array, it reconstructs the actual values efficiently. This way, instead of updating nums repeatedly, we can process all queries in an optimized manner and then traverse nums just once to check if all elements have become zero.

Now that we optimized how we apply queries, the next step is to determine how many queries we actually need. Instead of processing all queries one by one, we can use **binary search** to quickly determine the minimum number of queries required to achieve the zero array. We start by setting two pointers: left = 0 and right = len(queries), representing the search range. The middle index, mid = (left + right) / 2, represents the number of queries we

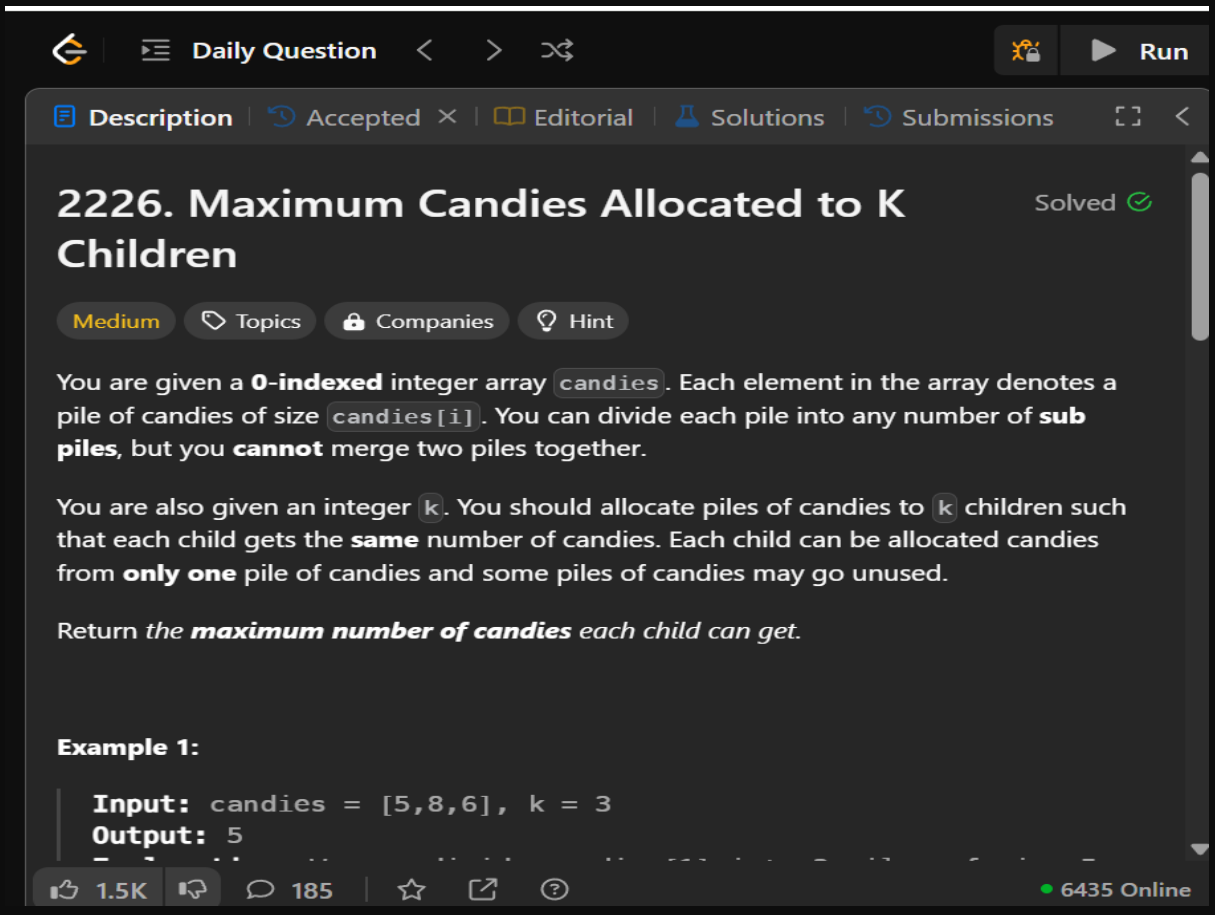
will attempt to apply. We update nums using only the first mid queries, compute the final state using the prefix sum of the difference array, and check if nums is now a zero array.

If it is possible to achieve a zero array with mid queries, we reduce our search range by setting right = mid - 1, since we might be able to do it with even fewer queries. Otherwise, we increase our search range by setting left = mid + 1, since we need more queries to reach the desired state. This binary search ensures that instead of checking every possible number of queries linearly $O(N)$, we find the answer in $O(\log N)$ time.

```cpp
class Solution {
public:
    int minZeroArray(vector<int>& nums, vector<vector<int>>& queries) {
        int left = 0;
        int n = nums.size();
        int right = queries.size();
        if(!Canzeroarray(nums,queries,right))return -1;
        while(left<=right){
            int middle = left+(right-left)/2;
            if(Canzeroarray(nums,queries,middle)){
                right = middle - 1;
            }
            else{
                left = middle + 1;
            }
        }
        return left;
    }
private:
    bool Canzeroarray(vector<int>&nums, vector<vector<int>>&queries,int k){
        int n = nums.size();
        vector<int>difference(n+1);
        int sum = 0;
        for(int i = 0; i<k; i++){
            int l = queries[i][0];
            int r = queries[i][1];
            int val = queries[i][2];
            difference[l] += val;
            difference[r+1] -= val;
        }
        for(int j = 0; j<n; j++){
            sum += difference[j];
            if(sum<nums[j]){
                return false;
            }
        }
        return true;

    }
```

For this problem:https://leetcode.com/problems/zero-array-transformation-ii/editorial




```cpp
class Solution {
public:
    int maximumCandies(vector<int>& candies, long long k) {
        if(!possibletogive(candies,k))return 0;
        int high = *max_element(candies.begin(),candies.end());
        int low = 1;
        int result;
        while(low<=high){
            int mid = (low+high)/2;
            if(cangivecandies(candies,mid,k)){
                result = mid;
                low = mid+1;
            }
            else{
                high = mid-1;
            }
        }
        return result;
    }
```

```cpp
    bool cangivecandies(vector<int>&candies,int b,long long k){
        long long count = 0;
        int n = candies.size();
        for(int i = 0; i<n; i++){
           count += candies[i]/b;
        }
        if(count>=k){
            return true;
        }
        else{
            return false;
        }
    }
    bool possibletogive(vector<int>&candies,long long k){
        long long sum = 0;
        int n = candies.size();
        for(int i = 0; i<n ; i++){
            sum = sum + candies[i];
        }
        if(sum>=k){
            return true;
        }
        else{
            return false;
        }
    }
};
```

This question:https://leetcode.com/problems/maximum-candies-allocated-to-k-children/description/?envType=daily-question&envId=2025-03-14

713. Subarray Product Less Than K

Solved

Medium | Topics | Companies | Hint

Given an array of integers `nums` and an integer `k`, return *the number of contiguous subarrays where the product of all the elements in the subarray is strictly less than* `k`.

Example 1:

> **Input:** nums = [10,5,2,6], k = 100
> **Output:** 8
> **Explanation:** The 8 subarrays that have product less than 100 are:
> [10], [5], [2], [6], [10, 5], [5, 2], [2, 6], [5, 2, 6]
> Note that [10, 5, 2] is not included as the product of 100 is not strictly less than k.

Example 2:

```cpp
int numSubarrayProductLessThanK(vector<int>& nums, int k) {
    if(k<=1)return 0;
    int n = nums.size();
    int count = 0 , l = 0, r = 0;
    long long product = 1;
    while(l<n && r<n){
        if(nums[r]>=k){
            r++;
            l = r, product = 1;
        }
        else{
            product = 1LL*product*nums[r];
            if(product<k){
                count += r-l+1;
                r++;
            }
            else{
                while(product>=k && l<n && l<r){
                    product /= nums[l];
                    l++;
                    if(product<k){
                        count += r-l+1;
                        r++;
                        break;
                    }
                }
                if(product>=k)product = 1;
            }
        }
    }
    return count;
```